Supplementary Material for "Evaluating GPT-5.4-mini for Gherkin Test Generation: An Empirical Study"

Nguyen Minh Hoang[1], Nguyen Hieu An[1], and Le Thi Quynh Chi[1*]

Department of Software Engineering, FPT University, Ho Chi Minh City, Vietnam
Email: hoangminh6112006@gmail.com (N.M.H.); hieuannguyen2k6@gmail.com (N.H.A.); chiltq6@fe.edu.vn (L.T.Q.C.)
[*] Corresponding author

This supplement belongs to the paper above. Section, table, figure and reference numbers without the prefix S refer to the paper; the references are those of the paper.

S1 Prompt, Skeleton Example and Additional Results

Listing S1 gives the user prompt and Listing S2 the system messages (Section **??**), Table S1 maps each project to its exemplars, Table S2 illustrates the skeleton extraction of Section **??**, Fig. S1 and Fig. S2 show the distribution of skeleton cosine in C0 overall and per project, Table S3 gives the reference-point sensitivity analysis of Section **??**, and Table S4 gives the parser results of RQ2.

Listing S1. User prompt of conditions C0 and C2, verbatim except for the three braced placeholders; the first line ends with a space. C1 drops the second line and both example blocks.

```
Generate a BDD Gherkin file for the following user story.
Follow the exact style, wording, and conventions shown in the Example.

EXAMPLE USER STORY:
{example_user_story}

EXAMPLE GHERKIN:
{example_gherkin}

---
NOW GENERATE FOR THIS USER STORY:
{user_story}
```

Listing S2. System messages, verbatim. Each project keeps its message in C0, C1 and C2.

```
Sylius: You are an expert QA Engineer for Sylius, an E-commerce platform
    using the Behat framework. You write business-focused Gherkin tests.
    Focus on product variants, shopping carts, and UI interactions using
    business language.

Apache Fineract: You are an expert QA Engineer for Apache Fineract, a
    core banking system. Your signature style is using detailed Data
    Tables in the Background block to mock extensive financial data (e.g.
     LoanProduct, clients). Write highly technical, data-driven Gherkin.

Diaspora: You are an expert QA Engineer for Diaspora, a Ruby on Rails
    social network using Capybara and Cucumber. Your signature style is
    writing concrete UI automation steps using CSS selectors (e.g. '
    within ".class"') and exact UI actions.
```

Table S1. Exemplar per project and condition (dataset id and reference file). The example story is the dataset story of that id and the example Gherkin is its full reference file.

Project	Condition	Id	Reference file
Sylius	C0	2	`being_unable_to_see_cart.feature`
	C2	86	`receiving_discount_based_on_total_of_items_from_specific_taxon.feature`
Fineract	C0	102	`WorkingCapitalNearBreachConfiguration.feature`
	C2	119	`LoanInterestPaymentWaiver.feature`
Diaspora	C0	202	`media-embed.feature`
	C2	203	`connects_users.feature`

The replication package holds the components of every prompt as separate files in the folder `revision/prompts/`: the three system messages of Listing S2, the one-shot template of Listing S1, the zero-shot template of C1, the story and reference Gherkin of each of the six exemplars (subfolder `exemplars/`), and the mapping of Table S1 (`exemplar_mapping.csv`). A C0 or C2 prompt is the one-shot template with the story and Gherkin of the project's exemplar in place of `{example_user_story}` and `{example_gherkin}`. The script `revision/prompts/make_prompt_artifacts.py` rebuilds these files from the dataset and the generation code without calling the API. It checks that the system messages equal those of the original notebook and that the C0 prompts it assembles equal those of the notebook for all 261 stories. All files use LF line breaks, and their SHA-256 hashes are listed below (in `MANIFEST_SHA256.txt` the exemplar files carry the prefix `exemplars/`). C0 was run on 19 July 2026 and C1 and C2 on 16 October 2026, as recorded in the API logs. In those logs the prompt-token difference between C0 or C2 and

C1 is the same for every file of a project (C0: 271, 1,910 and 299 tokens for Sylius, Fineract and Diaspora; C2: 960, 9,413 and 965), which is consistent with one fixed exemplar block per project. The version used for this paper is the release tag `fisat2026-camera-ready`, https://github.com/simonhoang611/RT-SWT-004-Group4/tree/fisat2026-camera-ready.

`exemplar_mapping.csv`
`982a690189c23702e9f8ac7b656a9072eab1af252a3b3ba711966c5432222b75`

`ex_102.feature`
`3125864ffcac5dcb98c849b4d138771005ee1ad8405e7154f4ae9463d4388222`

`ex_102_user_story.txt`
`58d9a3af1effd231652d7866f3dccabfc7c5ff4a6cce48816ae963f39558c4d0`

`ex_119.feature`
`bf6699e8e799aa5cf4f850030bc22338c72fd85f85972076a30ba7e538b5dc51`

`ex_119_user_story.txt`
`db4a5a90db39985ae0dba288f1148ce783f661e93da038543d6257a072ab60c0`

`ex_2.feature`
`f24b39313e8729427c4a57f87a829015947e75d3f01f2ec84ec2b7cc477530cc`

`ex_202.feature`
`1ef601612044212a7497f9cd7783efda22227b245355e848f999b5ca9ee26c7b`

`ex_202_user_story.txt`
`db47c0704403977bb012616803c90ef5b7544bc16fe7983234846912e83a09ae`

`ex_203.feature`
`358d7055e7b9d35685a83ac3b7e833dbd6bbf3d33c9ed06284825dd16b4fc5b5`

`ex_203_user_story.txt`
`06513657140c86fdb09325892bdcb46779e62b5a165c6dc6931d49d1e512b242`

`ex_2_user_story.txt`
`a87eb54d5e418bd8b03d9492ba3d589edc0063ac08b36246bd6ff819410392be`

`ex_86.feature`
`2a8802083c191d1d22f3d9d96185f48602dfe7827945124e4f100f42d998c193`

`ex_86_user_story.txt`
`c20891f56ce4fdb8598f4079a3a94c0dd94a51bc0a8b2be430246afc8313eebd`

`system_message_ecommerce.txt`
`64456030e3229cf63ce9f23a574433e67bc707864dd365e6aca842cab0c82498`

`system_message_finance.txt`
`2193526c6d69c6a09c8f6a34d3a37eafe3ea3f00d0604a47f85b47b7b7c27afd`

`system_message_social.txt`
`24bd9e8f470cc8f5fae0ea4d88b6413d70fd473ed0e4e25dfa75a197c56c19b9`

`user_prompt_one_shot_template.txt`
`e8908eb5441c9e8c8d3f15136d8295d9b011690a0f99b5d3a9d98356829d6387`

`user_prompt_zero_shot_template.txt`
`95645cbaf0b8fae23aa33b68137a0f95d02c626c26db2c91533d097fb6a05364`

Table S2. Skeleton extraction on a short Sylius-style fragment.

Original line	Skeleton line
`@ui`	(removed)
`Given a product "T-Shirt" priced at "$20.00"`	`Given a product <PARAM> priced at <PARAM>`
`When I add 3 units to the cart`	`When I add <NUM> units to the cart`
`| name | price |`	(removed)
`Then the total should be "$60.00"`	`Then the total should be <PARAM>`

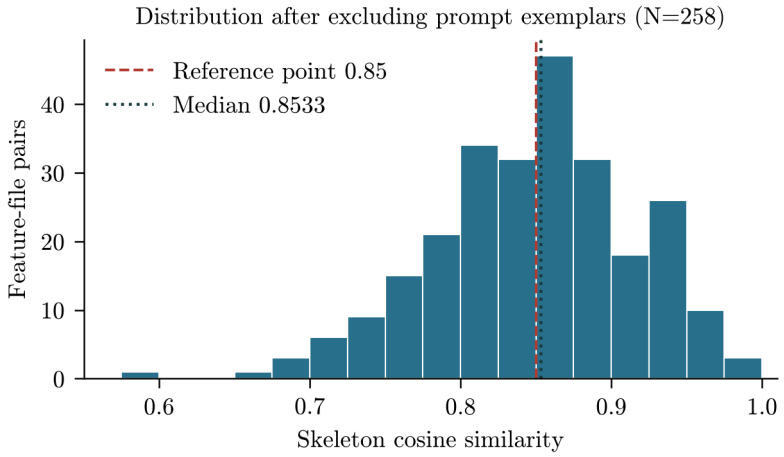


Fig. S1. Distribution of skeleton cosine similarity for the 258 non-exemplar files (condition C0).

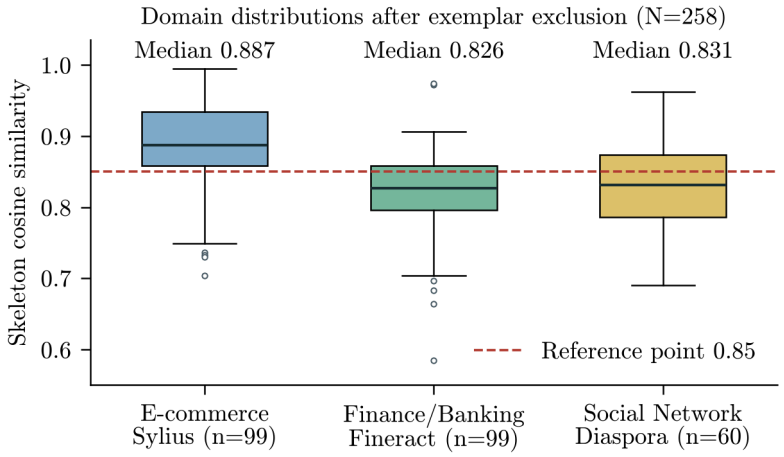


Fig. S2. Skeleton cosine similarity per project (C0, $N = 258$). The dashed line marks 0.85.

Table S3. Reference-point sensitivity (C0, $N = 258$). The p-value is the one-sided Wilcoxon test that the median exceeds t; the exact sign test gives the same decisions at every t ($p = 0.002$ at 0.84 and 0.209 at 0.85).

t	Share $\geq t$ [95% CI]	Median $-t$	p	Sylius	Fineract	Diaspora
0.80	78.3% [72.8, 83.2]	+0.053	< 0.001	91.9%	73.7%	63.3%
0.82	67.8% [61.8, 73.5]	+0.033	< 0.001	87.9%	56.6%	53.3%
0.84	59.3% [53.0, 65.4]	+0.013	0.010	84.8%	40.4%	48.3%
0.85	52.7% [46.4, 58.9]	+0.003	0.474	77.8%	34.3%	41.7%
0.86	43.4% [37.3, 49.7]	−0.007	0.988	73.7%	22.2%	28.3%
0.88	30.6% [25.1, 36.6]	−0.027	1.000	55.6%	12.1%	20.0%
0.90	22.1% [17.2, 27.7]	−0.047	1.000	45.5%	4.0%	13.3%

Table S4. Parser compliance of condition C0 (behave 1.3.3 parser).

Statistic	$N = 258$ (primary)	$N = 261$
Parser PASS / FAIL	255 / 3	258 / 3
ESR	98.84%	98.85%
95% Clopper–Pearson CI	[96.64%, 99.76%]	[96.68%, 99.76%]
Exact binomial p ($p_0 = 0.80$)	4.6×10^{-21}	2.4×10^{-21}

S2 Perturbation Test

Design. The perturbation test measured whether the metric reacts to seeded defects. From the reference files, 120 scenarios (40 per project, fixed seed) were extracted, and eight operators were applied where applicable: changing a number, a quoted value, a comparison operator or a negation, dropping a Then or a Given step, swapping a When with a Then, and moving a Then to the front. Each mutant was compared with its original scenario using the same skeleton metric.

Results. The 120 extracted scenarios yielded 683 applicable mutants. Of these, 29 mutants do not parse, and the extraction omitted the Background of 74 of the 120 source scenarios. Restricting to the 623 pairs in which both texts parse and the original has no keyword-order flag, every mutant keeps a skeleton cosine of at least 0.8995 with its original. At 0.90, 622 of 623 mutants would still pass, and at 0.95, 97.9%. For changed numbers and quoted values (179 pairs) the skeletons are identical, which follows from masking and is not an empirical finding. Swapped When/Then steps and Then steps moved to the front (210 pairs) score at least 0.954, although the keyword-order check flags all of them. Because the test compares single scenarios and the main metric compares whole files, these results show insensitivity at the scenario level and do not estimate a false-negative rate for whole files.

S3 Further Method and Result Details

This appendix holds the details that Sections **??**, **??**, **??** and **??** summarise.

Comparison with related studies. Table S5 compares this study with the five BDD-generation studies of Section **??**.

Unit of analysis. The unit of analysis is a feature file, not a scenario. With the behave 1.3.3 parser, the 244 parseable files among the 258 analysed references contain 2,554 expanded scenarios, and the 255 parseable generated files contain 1,913. The reference files were not screened with a parser before inclusion: 14 of the 261 reference files fail to parse with behave 1.3.3 (13 with behave 1.2.6), 13 of them because their stored text is cut (see below); the full upstream files of these 13 parse. They remain in the dataset, because the metric of Section **??** does not require a parse.

Sources and selection. The files were collected in two batches. A random sample of 100 files over the three projects (34, 33 and 33) was downloaded on 28 June 2026. No clone of that download was kept, but the Fineract files that match an upstream version date it to the develop branch between 23 and 29 June 2026. To reach 100 files per project, 161 further files were drawn at random from the clones of Table **??**; the random seed was not recorded. Diaspora's 71 feature files have only 58 distinct names, because ten names exist in both its desktop and mobile folders; the 61 selected files cover all 58 names. In the first batch the stored reference differs from the upstream file in two ways: the free-text Feature description is missing from 53 Sylius and Diaspora files, which the skeleton drops anyway, and 23 Fineract references end after 30,038 characters

Table S5. Comparison with BDD-generation studies, as reported in each paper. The line under each study gives its focus, models and prompting, and main finding.

Study	Data and openness	Syntax oracle	Semantic oracle	Human defect labels	Metric perturb.	Exemplar varied
Rathnayake [1]	500 stories, one company; released	none	BERTScore and other similarity, LLM judge	1–5 quality ratings; no defect labels	no	no
	Focus: best model, prompt and input for scenario generation. *Models:* GPT-4, Claude 3, Gemini; zero-shot, few-shot, chain-of-thought. *Finding:* GPT-4 scored highest on similarity, Claude 3 highest with experts and LLM judges; the best prompt differed by model.					
Karpurapu [2]	about 50 public stories; release not stated	Gherkin-lint	none	none	no	no
	Focus: syntax errors and validation accuracy of generated acceptance tests. *Models:* GPT-3.5, GPT-4, Llama-2-13B, PaLM-2; zero-shot, few-shot. *Finding:* GPT-3.5 and GPT-4 produced error-free tests; few-shot prompts gave fewer syntax errors.					
Ferreira [3]	13 stories, one company; confidential	TypeScript check of Cypress code	manual relevance check; helpfulness ratings	three error types in 20 tests	no	no
	Focus: Gherkin and Cypress generation in a company workflow. *Models:* GPT-4 Turbo; one iteratively engineered prompt per step. *Finding:* users found 95% of the scenarios helpful; 60% of the Cypress tests reflected their scenarios.					
Fernandes [4]	10 of 1,286 cases, one system	none	METEOR	qualitative error types	no	no (fixed format)
	Focus: accuracy and stability on free-form test descriptions. *Models:* seven LLMs, e.g. GPT-4 Turbo, Gemini, DeepSeek R1; zero-, one-, few-shot. *Finding:* Gemini had the highest mean METEOR (0.84), but model differences were not significant.					
Fonseca [5]	13 issues, one app; excerpts only	manual keyword check	LLM validator; practitioner survey	none	no	not stated
	Focus: Gherkin, Page Objects and UI tests from issues and code changes. *Models:* local DeepSeek-R1 for Gherkin, DeepSeek-Coder-V2, Gemma3:1b. *Finding:* 93.3% of the scenarios were syntactically valid and all generated UI tests ran.					
This study	261 files, three public projects; released	behave 1.3.3 parser	none; structural skeleton cosine plus defect audit	criterion-level audit of 60 files; agreement and consensus on 15	8 operators, 623 pairs	yes (C1, C2)
	Focus: RQ1 and RQ2. *Model:* gpt-5.4-mini-2026-03-17; same-project one-shot (C0), zero-shot (C1), other exemplar (C2). *Finding:* 98.8% parse; median similarity 0.853 is not above 0.85; high-similarity files still contain defects.					

of stored text. Scoring against the full upstream files changes only files 114 and 133, both of which stay below 0.85, so the median, the sign test and every RQ1 decision are unchanged (Wilcoxon $p = 0.452$ instead of 0.474). Extraction paths, clone records and the per-file comparison are in the replication package.

Data flow. Fig. S3 shows how the files pass through generation and analysis, with the denominator of each analysis.

RQ1 test details. The sign test is a binomial test of the number of files above against the number below 0.85 with ties dropped (none occurred), and the bootstrap interval uses 10,000 resamples with seed 2026. Symmetry is checked with the skewness of $x_i - 0.85$ and its bootstrap interval. δ is the sign-test

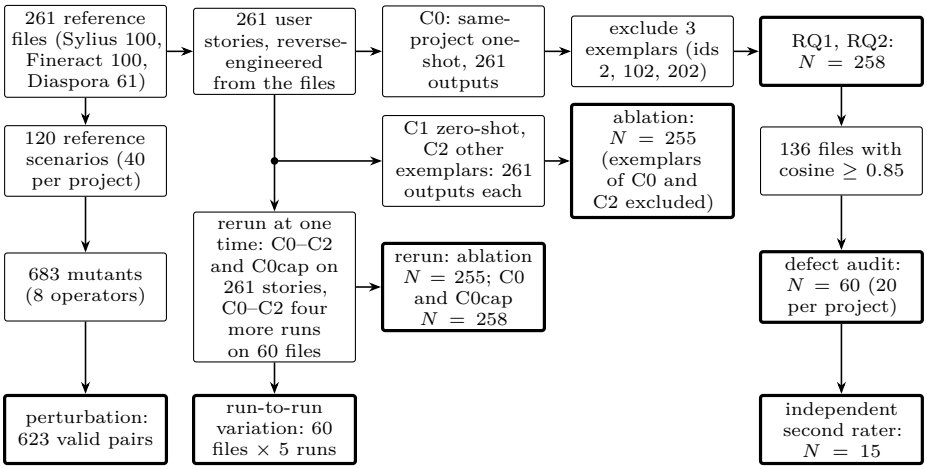


Fig. S3. Data flow and the denominator of each analysis. Bold boxes are analysis sets. Exemplars are excluded so that no prompt example is scored against itself. The rerun repeats the prompts within one time window.

statistic expressed as a proportion and a one-sample analogue of Cliff's δ against the reference point. Quartiles use linear interpolation and the SD uses $n - 1$.

Encoder for the ablation. Because the environment used for this analysis could not download the encoder, skeleton cosine for all three conditions was computed with an ONNX export of all-MiniLM-L6-v2 using the same tokenisation, 256-token truncation, mean pooling and normalisation; it reproduces the original C0 scores to within 4×10^{-7}, and a rerun with sentence-transformers (model revision `1110a24`) agrees with it to within 3×10^{-7} in all three conditions.

Rerun analysis plan. On the 60 files it reports, per condition, the within-file standard deviation (SD) of cosine across runs, the intraclass correlation ICC(1,1) and the number of files whose parser outcome or side of 0.85 changes; for C0 and C2 it adds a two-way analysis of variance (file random, prompt fixed), the mean between-prompt effect with a paired t interval over files, and a Welch interval per file.

Failure handling. The pipeline loads the stories, builds the prompt for the story's project and calls the API. It removes a Markdown code fence only when the model wraps its output in one; no generated text is edited by hand. Failure handling differed between runs. The C0 notebook set no timeout, so the client library's default of 600 s applied, and retried any failed request, including rate-limit errors, after a 60 s wait with no limit on the number of attempts; it did not check for empty output and logged only successful calls. For C1 and C2 a script made up to five attempts per story after an error, waiting 5, 10, 20, 40 and 60 s; it marked an output that was empty after fence removal, or a story whose five attempts all failed, as not OK with an empty output, to be retried in a later run of the script; such a story would have stayed in the denominator as a parser FAIL

and been excluded from the cosine analysis. In both runs the client library could also retry a request twice on its own. None of these events is recorded: all 522 C1 and C2 requests succeeded at the first attempt, no output in any condition was empty, and all 261 C0 requests returned an output. The intervals between consecutive C0 log entries track output length (Pearson $r = 0.995$, at most 61 s), so a 60 s retry wait is unlikely to have occurred. The rerun script set a 600 s timeout and disabled the library's retries, so that every attempt is logged; it made up to five attempts per request, waiting 5, 10, 20 and 40 s, and counted an empty output as a failed attempt; a request still failing after a resumed run would have stayed in the denominator as a parser FAIL. All 1,764 rerun requests succeeded at the first attempt with a non-empty output. Each call is appended to a log with its timestamp and token counts. In the C0 log, the model field is the requested model name written by the script, not the name returned by the API, and the logged cost was computed with outdated prices. Recomputed from the logged token counts (328,257 prompt and 380,494 completion tokens) at the Standard list price of $0.75 and $4.50 per million input and output tokens, without cache discounts, the estimated C0 cost is approximately $1.96 (an estimate from the logged tokens, not an invoice). The C1, C2 and rerun calls log the model name returned by the API; it was the requested snapshot in every call.

Defect facets. For each criterion the labeller recorded two facets. The business-logic facet covers wrong or missing numbers, operators, negations and conditions, wrong expected outcomes, untested criteria and contradicting additions. The temporal/causal facet covers missing preconditions, wrong triggering actions, missing or non-causal outcomes and impossible state orderings.

Keyword-order check. The automated check parses each file and flags four keyword-order violations per scenario: no Then step, a Given after a When or Then, a Then before any When, and a non-Given step in the Background. These flags encode the checker's conventions and do not by themselves establish a business defect.

RQ1 descriptive statistics. Table S6 gives the distribution of skeleton cosine in condition C0 for both denominators.

Skew of the C0 scores. The lower half of the interquartile range is wider than the upper (0.048 against 0.037), and the mean (0.848) lies below the median (0.853).

Truncated story input. The 84 Fineract files whose stories saw only the first 15,000 characters of the reference do not score lower than the 15 whose stories saw all of it (median 0.827 vs. 0.824; 27 of 84 and 7 of 15 reach 0.85; two-sided Mann–Whitney $p = 0.99$). Rescoring the 84 files against the skeleton of that 15,000-character prefix changes one score, by -0.004, because the encoder reads only 256 tokens and 82 of the 84 prefix skeletons are longer; the Fineract and pooled medians stay at 0.826 and 0.853. With only 15 non-truncated files this comparison is descriptive.

Reference point and defects. We could not calibrate the reference point against defect labels. The defect audit (Section **??**) sampled only files at or above 0.85, so it cannot estimate how often files below a given reference point are

Table S6. Skeleton cosine similarity of condition C0. The primary analysis excludes the three prompt exemplars.

Statistic	$N = 258$ (primary)	$N = 261$
Minimum	0.585	0.585
Q1	0.805	0.806
Median	0.853	0.854
Q3	0.890	0.894
Maximum	0.994	1.000
Mean ± SD	0.848 ± 0.068	0.850 ± 0.069
Median 95% bootstrap CI	[0.844, 0.859]	[0.845, 0.861]
Files ≥ 0.85	136 (52.7%)	139 (53.3%)
Sign test, one-sided p	0.209	0.161
Wilcoxon W, p (sensitivity)	16,784, 0.474	17,548, 0.355
Skewness [95% CI]	−0.41 [−0.76, −0.08]	−0.33 [−0.68, −0.01]
δ [95% CI]	0.054 [−0.072, 0.179]	0.065 [−0.060, 0.189]

defective. Within the audited range, cosine did not differ significantly between defective and non-defective files (Mann–Whitney $p = 0.21$; AUC 0.40 when a higher cosine is read as predicting a defect, i.e. defective files tended to score slightly lower). With 60 files from a restricted range, this neither shows nor rules out that the metric discriminates. Defects occurred in 11 of 24 files between 0.85 and 0.875, 10 of 16 between 0.875 and 0.90, 2 of 11 between 0.90 and 0.95, and 2 of 9 at 0.95 or above.

Example of a defect. For example, a criterion in the story of Sylius file 76 expects a cart total of $73.80 after "one taxed product" is removed from a cart holding three units of a $100 product and two units of a $30 product, both taxed at 23%. That total corresponds to removing the $100 product entirely. The generated scenario removes a single unit and still expects $73.80, although its own data imply $319.80. The file scores 0.914, and both the reviewed labels and the independent rater mark it as defective.

Consensus round. Of the six disagreements across the two facets, five concern three Diaspora files that the independent rater marked as defective and the reviewed labels did not; the sixth is a Sylius file in the opposite direction. All three Fineract files received the same labels. In the consensus round (28–29 October 2026), the two raters kept the shared label in all 24 agreed decisions and adopted the independent rater's label in all six disagreements, so the three Diaspora files became defective and the Sylius file lost its business-logic defect.

Defect types. The most frequent business-logic defects are wrong expected outcomes (20 files), missing or wrong conditions (9) and wrong numbers (8); no wrong operator or lost negation was found. On the temporal/causal facet, outcomes without a verifying Then are most frequent (19 files), followed by Then steps that do not follow from the When (9) and missing preconditions (8).

Parser failures of C0. The three failures (files 136, 148 and 154) are an Examples block outside a Scenario Outline in two files and a malformed data table in one.

Output length in the ablation. In the same-time rerun, outputs were longer under C1 and C2 than under C0 (median 750 and 752 completion tokens against 498).

Original ablation runs. Table S7 gives the ablation of Table **??** in the original runs, with C0 run on 19 July and C1 and C2 on 16 October 2026.

Table S7. Prompt ablation in the original runs (C0 on 19 July, C1 and C2 on 16 October 2026), on the same 255 files and with the same columns as Table **??**.

Condition	Median	≥ 0.85 [95% CI]	Sylius	Fineract	Diaspora	PASS
C0	0.853	52.9% [46.6, 59.2]	0.886	0.827	0.835	252
C1	0.819	34.5% [28.7, 40.7]	0.873	0.769	0.825	252
C2	0.875	64.3% [58.1, 70.2]	0.892	0.876	0.840	254

Run-to-run details. On the 60 repeated files the five outputs were identical for one file under C0 and for none under C1 or C2. The 95% CIs of ICC(1,1) are 0.888–0.948 for C0, 0.864–0.936 for C1 and 0.898–0.953 for C2. Between the new and the original C0, 2 of 258 outputs are identical and the median per-file difference is 0.000 (IQR −0.018 to 0.014). The new C0 has a 95% CI of 0.843–0.862 for the median, with 135 files above 0.85 and 123 below.

Variance components of the rerun. For C0 and C2, the within-prompt SD is 0.020, the file-by-prompt SD 0.042 and the between-file SD 0.055. Fineract shows the largest mean shift from C0 to C2 (+0.013; 9 per-file intervals above 0, 6 below).

Output cap. Under C0cap, 81 of the 258 non-exemplar outputs stopped at the cap, and 239 files passed the parser (92.6%, 95% CI [88.7%, 95.5%]) against 258 for the rerun C0 (exact McNemar $p < 0.001$); all 19 failures are capped outputs. The median cosine was 0.854, with a median paired difference of +0.0005 to the rerun C0 (Wilcoxon $p = 0.108$). These two tests are descriptive and uncorrected.

Details of the threats to validity. The stories were written by GPT-5.4-mini; story–reference trigram overlap has a median of 4.9% (Section **??**), and 84 Fineract stories were written from a 15,000-character prefix and score no lower than the other Fineract files (Section **??**). Structural leakage such as the order and number of criteria is not measured, and no independently written backlog was available. The original ablation ran three months after C0; the C0 of the same-time rerun has almost the same median as the original C0 (0.852 against 0.853), but the rerun cannot show what changed in the hosted service between the two dates. The returned model name was recorded for C1, C2 and the rerun, the original C0 log records only the requested name, and the API returned no system fingerprint. For about two thirds of the files the similarity metric sees only the first 256 tokens of a skeleton, and a valid alternative scenario can score low because it differs from the single project-maintained reference. The audit labels were drafted by an AI assistant and retained unchanged by a

reviewing author, whose review was recorded as one confirmation rather than per-file decisions; the independence of the 15-file rater rests on the rater's own declaration, and the κ intervals reach down to 0.05 and 0.24. No rater who had not seen the labels was available to relabel the other 45 files; because the consensus adopted the independent rater's label in every disagreement and added defects in three Diaspora files, their reviewed labels may undercount defects. The two label facets are not independent. The perturbation test mutates single extracted scenarios, omitted the Background of 74 of 120 source scenarios, and draws several mutants from the same scenario, so its pairs are not independent. The projects contribute 99, 99 and 60 analysed files, and results may differ for short tickets, mixed-language requirements, private conventions or projects with unstable step definitions.

References

1. Rathnayake, A., Shahin, M., Abaei, G.: Behaviour Driven Development Scenario Generation with Large Language Models. arXiv:2603.04729 (2026). https://doi.org/10.48550/arXiv.2603.04729
2. Karpurapu, S., Myneni, S., Nettur, U., Gajja, L.S., Burke, D., Stiehm, T., Payne, J.: Comprehensive Evaluation and Insights Into the Use of Large Language Models in the Automation of Behavior-Driven Development Acceptance Test Formulation. IEEE Access 12, 58715–58721 (2024). https://doi.org/10.1109/ACCESS.2024.3391815
3. Ferreira, M., Viegas, L., Faria, J.P., Lima, B.: Acceptance Test Generation with Large Language Models: An Industrial Case Study. In: IEEE/ACM International Conference on Automation of Software Test (AST), pp. 1–11 (2025). https://doi.org/10.1109/AST66626.2025.00007
4. Fernandes, H., Perkusich, M., Albuquerque, D., Silva, I., Santos, D.F.S., Gorgônio, K., Perkusich, A.: A Comparative Study of LLMs for Gherkin Generation. In: Brazilian Symposium on Software Engineering (SBES), pp. 171–181 (2025). https://doi.org/10.5753/SBES.2025.9888
5. Fonseca, P.L., Lima, B., Faria, J.P.: Streamlining Acceptance Test Generation for Mobile Applications Through Large Language Models: An Industrial Case Study. In: 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 3311–3321 (2025). https://doi.org/10.1109/ASE63991.2025.00273
6. Zhang, Q., Shang, Y., Fang, C., Gu, S., Yu, S., Zhou, J., Chen, Z.: TestBench: Evaluating Class-Level Test Case Generation Capability of Large Language Models. Frontiers of Computer Science 21(6), 2106202 (2027), published online 13 October 2025. https://journal.hep.com.cn/fcs/EN/10.1007/s11704-025-50078-9, last accessed 2026/10/28.
7. Chang, H.-F., Shokrolah Shirazi, M.: A Systematic Approach for Assessing Large Language Models' Test Case Generation Capability. Software 4(1), 5 (2025). https://doi.org/10.3390/software4010005
8. Wang, Z., Wang, W., Li, Z., Wang, L., Yi, C., Xu, X., Cao, L., Su, H., Chen, S., Zhou, J.: XUAT-Copilot: Multi-Agent Collaborative System for Automated User Acceptance Testing with Large Language Model. arXiv:2401.02705 (2024). https://doi.org/10.48550/arXiv.2401.02705

9. Liu, J., Xia, C.S., Wang, Y., Zhang, L.: Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In: Advances in Neural Information Processing Systems 36 (NeurIPS 2023), pp. 21558–21572 (2023). https://doi.org/10.52202/075280-0943
10. Tong, W., Zhang, T.: CodeJudge: Evaluating Code Generation with Large Language Models. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 20032–20051 (2024). https://doi.org/10.18653/v1/2024.emnlp-main.1118
11. Kang, S., Yoon, J., Yoo, S.: Large Language Models are Few-shot Testers: Exploring LLM-based General Bug Reproduction. In: 45th IEEE/ACM International Conference on Software Engineering (ICSE), pp. 2312–2323 (2023). https://doi.org/10.1109/ICSE48619.2023.00194
12. Fakhoury, S., Naik, A., Sakkas, G., Chakraborty, S., Lahiri, S.K.: LLM-Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation. IEEE Transactions on Software Engineering 50(9), 2254–2268 (2024). https://doi.org/10.1109/TSE.2024.3428972
13. Harman, M., Ritchey, J., Harper, I., Sengupta, S., Mao, K., Gulati, A., Foster, C., Robert, H.: Mutation-Guided LLM-based Test Generation at Meta. In: Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion), pp. 180–191 (2025). https://doi.org/10.1145/3696630.3728544
14. Zadenoori, M.A., Dąbrowski, J., Alhoshan, W., Zhao, L., Ferrari, A.: Large Language Models (LLMs) for Requirements Engineering (RE): A Systematic Literature Review. arXiv:2509.11446 (2025). https://doi.org/10.48550/arXiv.2509.11446
15. Baltes, S., Angermeir, F., Arora, C., Muñoz Barón, M., Chen, C., Böhme, L., Calefato, F., Ernst, N., Falessi, D., Fitzgerald, B., Fucci, D., He, J., Treude, C., Kalinowski, M., Lambiase, S., Russo, D., Lungu, M., Martinez Montes, C., Prechelt, L., Ralph, P., van Tonder, R., Wagner, S.: Guidelines for Empirical Studies in Software Engineering involving Large Language Models. arXiv:2508.15503v8 (2026). https://doi.org/10.48550/arXiv.2508.15503
16. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: EMNLP-IJCNLP, pp. 3982–3992 (2019). https://doi.org/10.18653/v1/D19-1410
17. Sylius: Sylius, branch 2.3, commit 9ab28ca. https://github.com/Sylius/Sylius/tree/9ab28ca4b5ad5797216fe14e72e4d6cfe29c3fdb, last accessed 2026/10/28.
18. Apache Software Foundation: Apache Fineract, branch develop, commit 5a93304. https://github.com/apache/fineract/tree/5a93304677061501f5d0c243b8dbf84752e08b0b, last accessed 2026/10/28.
19. diaspora* contributors: diaspora*, branch develop, commit b7bf1dc. https://github.com/diaspora/diaspora/tree/b7bf1dc33566c036a3d788dd8876e97c30905842, last accessed 2026/10/28.
20. OpenAI: GPT-5.4 Mini model documentation. https://developers.openai.com/api/docs/models/gpt-5.4-mini, last accessed 2026/10/28.
21. OpenAI: Create chat completion, API reference. https://developers.openai.com/api/reference/resources/chat/subresources/completions/methods/create, last accessed 2026/10/28.

22. Sentence Transformers: all-MiniLM-L6-v2 model card, revision 1110a24. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2/tree/1110a243fdf4706b3f48f1d95db1a4f5529b4d41, last accessed 2026/10/28.
23. behave developers: behave v1.3.3 release. https://github.com/behave/behave/releases/tag/v1.3.3, last accessed 2026/10/28.
24. Cohen, J.: A Coefficient of Agreement for Nominal Scales. Educational and Psychological Measurement 20(1), 37–46 (1960). https://doi.org/10.1177/001316446002000104
25. Arcuri, A., Briand, L.C.: A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering. In: 33rd International Conference on Software Engineering, pp. 1–10 (2011). https://doi.org/10.1145/1985793.1985795
26. Agresti, A.: Categorical Data Analysis, 2nd edn. John Wiley & Sons (2002). https://doi.org/10.1002/0471249688
27. Efron, B., Tibshirani, R.J.: An Introduction to the Bootstrap. Monographs on Statistics and Applied Probability, vol. 57. Chapman & Hall, New York (1993)
28. Clopper, C.J., Pearson, E.S.: The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial. Biometrika 26(4), 404–413 (1934). https://doi.org/10.1093/biomet/26.4.404
29. Kruskal, W.H., Wallis, W.A.: Use of Ranks in One-Criterion Variance Analysis. Journal of the American Statistical Association 47(260), 583–621 (1952). https://doi.org/10.1080/01621459.1952.10483441
30. Holm, S.: A Simple Sequentially Rejective Multiple Test Procedure. Scandinavian Journal of Statistics 6(2), 65–70 (1979). https://www.jstor.org/stable/4615733